How Large Language Models (LLMs) Work — Internals Explained

1️⃣ What an LLM Really Is

- At its core, an LLM is a probability engine.
- Given a sequence of tokens, it predicts the next most likely token.

That's it.

- No consciousness. No understanding. Just extremely good statistical pattern matching at scale.

2️⃣ Text → Tokens (Input Processing)

- LLMs don't read words. They read tokens.

Example:

➔ "I love programming"
➔ → ["I", " love", " program", "ming"]

- Tokens are numbers (IDs)
- Vocabulary size ≈ 30k–100k tokens
- This allows models to handle any language efficiently

3️⃣ Embeddings: Turning Tokens into Math

- Each token ID is mapped to a vector (e.g., 4096 dimensions).
- Think of embeddings as:
- Meaning encoded as geometry.
- Similar meanings → vectors close together
- This is why LLMs "understand" synonyms

4️⃣ Transformer Architecture (The Game Changer)

- LLMs are built on Transformers.

Key components:

🔹 Self-Attention (The Secret Sauce)

- Each token asks: →“Which other tokens matter most to me right now?”
- This allows the model to:
    - Track context across long text
    - Understand relationships (cause/effect, subject/object)
- Attention formula (simplified):A
    - ttention(Q, K, V) = softmax(QK^T / √d) × V
- Translation: weighted focus, not magic.

🔹 Multi-Head Attention

Instead of one attention mechanism: →Multiple heads focus on different patterns

➔ Syntax
➔ Semantics
➔ Long-range dependencies
➔ Parallel thinking = stronger signal.

5️⃣ Feedforward Networks (Token Refinement)

- After attention: →Each token passes through dense neural layers
- Adds non-linearity
- Refines representation
- Think:
    - Attention finds what matters, feedforward decides what to do with it.

6️⃣ Stacking Layers (Depth = Power)

➔ A modern LLM has: →24 → 96+ transformer layers
➔ Each layer:
    ◆ Builds more abstract understanding
➔ Early layers → grammar
➔ Mid layers → meaning
➔ Deep layers → reasoning patterns

7️⃣ Training Phase (Where Intelligence Is Born)

- Step 1: Pre-trainingTrained on trillions of tokens
- Objective:

    - Predict the next token correctly
- Loss function:
    - Cross-entropy loss
- Back-propagation updates billions of parameters
- This phase teaches:
    - Language
    - FactsReasoning patterns
    - Code structure

- Step 2: Fine-TuningSupervised Datasets
    - Human-written answers
    - Task-specific behavior

- Step 3: RLHF (Human Alignment)Reinforcement Learning from Human Feedback:
    - Humans rank outputsModel learns preferencesOptimizes for usefulness, safety, clarity
    - This is why models feel “polite” and “helpful”.

8️⃣ Inference (When You Chat)

- When you type:
    - Text → tokens
    - Tokens → embeddings
    - Pass through transformer layers
    - Output probabilities for next tokenSampling strategy picks one:
        - TemperatureTop-k / Top-p
        - Repeat until done
        - No memory.
        - No thinking.
        - Just fast iteration.

9️⃣ Why LLMs Seem Intelligent

- Three reasons:
    - Massive data scale
    - Attention-based context modeling
    - Human-aligned fine-tuning
    - Emergent behavior, not explicit programming.

🔟 Hard Truths (No Sugar-Coating)

- LLMs don’t understand truth
- They don’t reason symbolically
- They hallucinate when probabilities lie
- They are only as good as:
    - Data
    - Prompts
    - Guardrails

TL;DR (Executive Summary)

- LLMs predict tokens
- Transformers enable context awareness
- Training creates emergent reasoning
- Prompt engineering = steering probabilities
- LLMs are stochastic parrots with PhDs—dangerous if misunderstood, powerful if controlled.